

TypeScript — знакомство с типами

TypeScript — это надстройка над JavaScript, добавляющая статическую типизацию и другие улучшения, которые делают код более предсказуемым и безопасным.



Зачем нужен?

- ✓ **Типизация:** обнаружение ошибок на этапе разработки, до выполнения кода.
- ✓ **Современные возможности JS:** поддержка ES6+ и транпиляция в совместимый код.
- ✓ **Лучший опыт разработки:** автодополнение, подсказки и рефакторинг в IDE.



Ключевые возможности TypeScript



Обнаружение ошибок
на этапе разработки

Меньше багов
в продакшене.



Интерфейсы
и типы

Структурирование
данных и контрактов.



Дженерики

Создание гибких
и переиспользуемых
компонентов.



Безопасность
и масштабируемость

Более стабильный
и поддерживаемый код.

TS

Типы данных в TypeScript

T

string

строка

```
let name: string = "React";
```

123

number

число

```
let age: number = 25;
```

✓

boolean

булево значение

```
let isActive: boolean = true;
```

*

any

любой тип

```
let value: any = 42;
```

⊘

null

пустое значение

```
let empty: null = null;
```

⊖

undefined

неопределенное значение

```
let u: undefined = undefined;
```

○

void

нет возвращаемого значения

```
function log(): void {}
```

{ }

object

объект

```
let obj: object = {};
```

</>

Пример

```
1 interface User {
2   id: number;
3   name: string;
4   isActive: boolean;
5 }
6
7 const user: User = {
8   id: 1,
9   name: "React Developer",
9   isActive: true
10 };
```



TypeScript помогает
писать более надёжный
и понятный код,
снижает количество
ошибок и ускоряет
разработку больших
приложений.



Типы данных в TypeScript

TypeScript предоставляет набор встроенных типов для описания данных в приложении.

Тип	Описание	Пример
 string	Строка	<code>let name: string = "Привет";</code>
 number	Число	<code>let age: number = 25;</code>
 boolean	Булево значение	<code>let isActive: boolean = true;</code>
 any	Любой тип	<code>let value: any = 'что угодно';</code>
 null	Пустое значение	<code>let empty: null = null;</code>
 undefined	Неопределенное значение	<code>let u: undefined = undefined;</code>
 void	Отсутствие возврата	<code>function log(): void {}</code>
 object	Объект	<code>let obj: object = {};</code>



Зачем нужны типы?



Раннее обнаружение ошибок

Ошибки находят на этапе разработки, а не в рантайме.



Безопасность кода

Защищает от неожиданных значений и багов.



Удобная разработка

Автодополнение, подсказки и рефакторинг в IDE.



Масштабируемость

Типы упрощают поддержку больших проектов.



Типизация и вывод типов

TypeScript может **автоматически определять тип переменной** на основе присваиваемого значения.

```
let title = "React";    // string
let count = 10;         // number
let isDone = false;    // boolean
```



Явная типизация рекомендуется для сложных данных и публичных API.



Сложные типы

• Массивы

```
let numbers: number[] = [1, 2, 3];
```

• Кортежи

```
let user: [string, number] = ["Alice", 30];
```

• Union (объединение)

```
let id: string | number = "abc";
```

• Literal (литеральные значения)

```
let direction: 'left' | 'right' = 'left';
```

Интерфейсы и типы помогают описывать структуру данных, делают код более понятным и упрощают работу в команде.

Постепенно переходите на строгую типизацию всего проекта.

Функции в TypeScript

TypeScript добавляет статическую типизацию к функциям, помогая избегать ошибок и делая код более понятным и поддерживаемым.

Синтаксис

```
function имя(параметр: тип): тип_возврата {  
  // тело функции  
}
```

Имя
функции

Параметры
с типами

Тип
возврата

- ✓ Типы параметров обязательны.
- ✓ Тип возврата можно опустить (TypeScript выведет сам).
- ✓ Можно указывать значение по умолчанию и необязательные параметры.

Специальные типы для функций

void

Функция ничего не возвращает.

```
function logMessage(message: string): void {  
  console.log(message);  
}
```

never

Функция никогда не возвращает значение (выбрасывает ошибку или завершает работу).

```
function throwError(message: string): never {  
  throw new Error(message);  
}
```

Примеры функций

1 Простая функция

```
function add(a: number,  
  b: number): number {  
  return a + b;  
}
```

```
const sum = add(2, 3);  
console.log(sum); // 5
```

2 Необязательный параметр

```
function greet(name: string,  
  greeting?: string): string {  
  return greeting  
    ? `${greeting}, ${name}!`  
    : `Привет, ${name}!`;  
}  
  
console.log(greet("Alice"));  
// Привет, Alice!  
console.log(greet("Alice", "Привет"));  
// Привет, Alice!
```

3 Параметр по умолчанию

```
function multiply(  
  a: number,  
  b: number = 1  
): number {  
  return a * b;  
}  
  
console.log(multiply(5)); // 5  
console.log(multiply(5, 3)); // 15
```

Стрелочные функции

```
const getFullName = (firstName: string,  
  lastName: string): string => {  
  return `${firstName} ${lastName}`;  
};  
  
console.log(getFullName("John", "Doe"));  
// John Doe
```



- ✓ Более короткий синтаксис.
- ✓ Не имеют собственного this.
- ✓ Идеальны для колбэков и небольших функций.

Best practices



Используйте явные типы параметров и возврата.



Названия функций должны чётко описывать их действие.



Делайте функции небольшими и выполняющими одну задачу.



Избегайте any — используйте более конкретные типы.



Используйте необязательные параметры осознанно.



Типизация функций делает ваш код надёжнее, упрощает работу в команде и улучшает автодополнение в IDE.



Классы в TypeScript

TypeScript поддерживает классы, наследование, модификаторы доступа и интерфейсы, что помогает создавать надёжные и масштабируемые приложения.



Основы классов

```
class User {
  id: number;
  name: string;

  constructor(id: number, name: string) {
    this.id = id;
    this.name = name;
  }

  greet(): string {
    return `Привет, ${this.name}!`;
  }
}

const user = new User(1, "Alice");
console.log(user.greet()); // Привет, Alice!
```

- ✓ Класс — это шаблон для создания объектов.
- ✓ Свойства описываются с указанием типа.
- ✓ Методы — функции, принадлежащие классу.
- ✓ Конструктор вызывается при создании объекта через `new`.



Интерфейсы и реализация

```
interface IUser {
  id: number;
  name: string;
  greet(): string;
}

class User implements IUser {
  constructor(
    public id: number,
    public name: string
  ) {}

  greet(): string {
    return `Здравствуйте, ${this.name}!`;
  }
}
```

- ✓ Интерфейс описывает контракт (набор свойств и методов).
- ✓ Класс, реализующий интерфейс, обязан предоставить их реализацию.
- ✓ Повышает надёжность и предсказуемость кода.



Совет

Для UI-компонентов в React чаще используются функции и хуки. Классы полезны для бизнес-логики, моделей и сложных сущностей.



Модификаторы доступа

Модификатор	Описание	Пример
<code>public</code>	Доступен из любого места (по умолчанию).	<code>public name: string;</code>
<code>private</code>	Доступен только внутри класса.	<code>private age: number;</code>
<code>protected</code>	Доступен внутри класса и его наследников.	<code>protected id: number;</code>



Наследование

```
class Animal {
  constructor(public name: string) {}

  move() {
    console.log(`${this.name} движется`);
  }
}

class Dog extends Animal {
  bark() {
    console.log("Гав-гав!");
  }
}

const dog = new Dog("Бобик");
dog.move(); // Бобик движется
dog.bark(); // Гав-гав!
```

- ✓ Класс `Dog` наследует от `Animal`.
- ✓ Наследуются свойства и методы.
- ✓ Можно добавлять новые методы и переопределять существующие.



Абстрактные классы

```
abstract class Shape {
  constructor(public color: string) {}

  abstract area(): number;
}

class Circle extends Shape {
  constructor(color: string, public radius: number) {
    super(color);
  }

  area(): number {
    return Math.PI * this.radius * this.radius;
  }
}
```

- ✓ Абстрактный класс нельзя создать напрямую (только наследовать).
- ✓ Абстрактные методы должны быть реализованы в дочерних классах.



Когда использовать классы?



Когда ваш код строится вокруг объектов и их поведения.



Когда нужно наследование и полиморфизм.



Когда требуется строгая структура и инкапсуляция.



Помните

TypeScript делает классы безопаснее, а ваш код — понятнее и легче в поддержке.



Итог

Классы, интерфейсы и наследование в TypeScript дают мощные инструменты для разработки надёжных приложений.

Продвинутые возможности TypeScript

TypeScript предлагает мощные инструменты для работы со сложными типами, дженериками и утилитарными типами.



Дженерики

Позволяют создавать переиспользуемые компоненты и функции с гибкими типами.

```
function identity<T>(value: T): T {
  return value;
}

const num = identity<number>(42); // 42
const str = identity<string>("hello"); // "hello"
```



Польза:
гибкость и переиспользование кода без потери типизации.



Утилитарные типы

Готовые типы помогают трансформировать и комбинировать типы.

Тип	Описание	Пример
<code>Partial<T></code>	Все свойства становятся опциональными	<code>Partial<User></code>
<code>ReadOnly<T></code>	Делает все свойства только для чтения	<code>ReadOnly<User></code>
<code>Pick<T, K></code>	Выбирает подмножество свойств	<code>Pick<User, "id" "name"></code>
<code>Omit<T, K></code>	Исключает указанные свойства	<code>Omit<User, "password"></code>
<code>Record<K, T></code>	Создаёт объект с ключами K и значениями T	<code>Record<string, number></code>



Условные типы

Позволяют выбирать тип в зависимости от другого типа.

```
type IsString<T> = T extends string
  ? "yes"
  : "no";

type A = IsString<string>; // "yes"
type B = IsString<number>; // "no"
```



Плезно для создания умных типов, которые адаптируются к входным данным.



Сложные типы

Объединяйте типы для описания сложных структур.

```
type ID = string | number;

type Status = "loading" | "success" | "error";

interface ApiResponse<T> {
  data: T;
  status: Status;
  error?: string;
}

const res: ApiResponse<User> = {
  data: { id: 1, name: "Alice" },
  status: "success",
};
```



Комбинируйте объединения, пересечения и дженерики для точного описания данных.



Ключевое слово **keyof**

Позволяет получить объединение ключей объекта.

```
interface User {
  id: number;
  name: string;
  email: string;
}

type UserKeys = keyof User; // "id" | "name" | "email"
```



Положите в основу функций, работающих с ключами объектов.



Оператор **satisfies** (с TypeScript 4.9)

Проверяет, что значение соответствует типу, не изменяя его тип.

```
const config = {
  apiUrl: "https://api.example.com",
  timeout: 5000,
} satisfies { apiUrl: string; timeout: number };

// Тип config: { apiUrl: string; timeout: number }
// Значение проверено на соответствие типу
```



Отлично подходит для конфигов и констант.



Декораторы (экспериментально)

Позволяют добавлять метаданные и модифицировать классы и их члены.

```
function log(target: any, key: string, descriptor: PropertyDescriptor) {
  const method = descriptor.value;
  descriptor.value = function (...args: any[]) {
    console.log(`Вызов ${key} c`, args);
    return method.apply(this, args);
  };
}

class Service {
  @log
  fetchData(url: string) {
    return `Данные из ${url}`;
  }
}
```



Используется в фреймворках и библиотеках (например, NestJS).



Лучшие практики



Включайте строгий режим (`strict`) в `tsconfig.json`



Избегайте `any` — используйте конкретные типы



Создавайте собственные типы и интерфейсы



Используйте утилитарные типы вместо дублирования



Держите типы рядом с данными и логикой



Итог

TypeScript — это не просто типы, это инструмент для создания надёжного, поддерживаемого и масштабируемого кода.



Полезные ресурсы



Официальная документация
<https://www.typescriptlang.org/docs>



TypeScript Handbook
<https://www.typescriptlang.org/docs/handbook>



Playground (редактор онлайн)
<https://www.typescriptlang.org/play>



Awesome TypeScript
<https://github.com/dzharil/awesome-typescript>



Продвинутые типы в TypeScript

TypeScript предоставляет мощные инструменты для работы со сложными типами и позволяет создавать гибкие и безопасные абстракции.

{ } Mapped Types

Преобразуют существующие типы.

```
type User = {
  id: number;
  name: string;
  email: string;
};

type ReadonlyUser = {
  readonly [K in keyof User]: User[K];
};
```

- ✓ `keyof` получает ключи объекта ("id" | "name" | "email").
- ✓ Проходим по ключам и преобразуем свойства.
- ✓ `Readonly` делает все свойства только для чтения.

Результат: `type ReadonlyUser = { readonly id: number; readonly name: string; readonly email: string; }`

Utility Types (полезные утилиты)

Тип	Описание	Пример
<code>Partial<T></code>	Все свойства становятся необязательными	<code>Partial<User></code>
<code>Required<T></code>	Делает все свойства обязательными	<code>Required<User></code>
<code>Pick<T, K></code>	Выбирает подмножество свойств	<code>Pick<User, "id" "name"></code>
<code>Omit<T, K></code>	Исключает указанные свойства	<code>Omit<User, "password"></code>
<code>Record<K, T></code>	Создает объект с ключами K и значениями T	<code>Record<string, number></code>
<code>ReturnType<T></code>	Тип возвращаемого значения функции	<code>ReturnType<typeof fn></code>
<code>Parameters<T></code>	Типы параметров функции (в виде кортежа)	<code>Parameters<typeof fn></code>
<code>ConstructorParameters<T></code>	Типы параметров конструктора	<code>ConstructorParameters<typeof C></code>
<code>InstanceType<T></code>	Тип экземпляра класса	<code>InstanceType<typeof C></code>

Типовые защиты (Type Guards)

```
function isString(value: unknown): value is string {
  return typeof value === "string";
}

function print(value: unknown) {
  if (isString(value)) {
    console.log(value.toUpperCase()); // value: string
  } else {
    console.log("Не строка");
  }
}
```

- ✓ Пользовательские гварды уточняют тип.
- ✓ Используются вместе с `typeof`, `instanceof`, `in`, или собственными функциями.
- ✓ `value is T` — специальный синтаксис для сужения.



Conditional Types

Выбирают тип на основе условия.

```
type IsString<T> = T extends string
  ? "yes"
  : "no";

type A = IsString<string>; // "yes"
type B = IsString<number>; // "no"
```

- ✓ Работают с дженериками.
- ✓ Позволяют создавать умные типы.
- ✓ Используются внутри утилитарных типов.



Пример из стандартной библиотеки:

`ReturnType<T>` — извлекает тип возвращаемого значения функции.



Discriminated Unions

Безопасные объединения с общим дискриминатором.

```
type Success = { type: "success"; data: string };
type Error = { type: "error"; error: string };
type Loading = { type: "loading" };
type Result = Success | Error | Loading;

function handleResult(res: Result) {
  switch (res.type) {
    case "success":
      console.log(res.data);
      break;
    case "error":
      console.error(res.error);
      break;
    case "loading":
      console.log("Loading...");
      break;
  }
  default:
    const _exhaustive: never = res; // проверка на полноту
  }
}
```

- ✓ Общий литерал (type) действует как дискриминатор.
- ✓ TypeScript сужает тип внутри ветви.
- ✓ Проверка полноты через `never` помогает не пропустить кейсы.

★ Рекомендуется для обработки состояний и ответов API.

JS Интеграция с JS

```
// @ts-check
/**
 * @param {string} name
 * @returns {string}
 */
function greet(name) {
  return `Привет, ${name}!`;
}

greet("TypeScript c JS");
```

- ✓ Можно использовать TypeScript в JavaScript через JSDoc.
- ✓ Добавляйте `// @ts-check` в начале файла.
- ✓ IDE будет давать подсказки и проверять типы.



Template Literal Types

Позволяют работать со строковыми литералами.

```
type EventName<T extends string> =
  `${T}Changed`;

type UserEvents =
  | EventName<"name">
  | EventName<"email">;

// "nameChanged" | "emailChanged"
```

- ✓ Комбинируйте строки.
- ✓ Создавайте события, роуты, ключи кеша и т.д.
- ✓ Поддерживаются встроенные типы (Uppercase, Lowercase и др.).

Полезные встроенные типы

<code>Uppercase<S></code>	Преобразует все символы в верхний регистр
<code>Lowercase<S></code>	Преобразует все символы в нижний регистр
<code>Capitalize<S></code>	Делает первую букву заглавной
<code>Uncapitalize<S></code>	Делает первую букву строчной



Readonly и const

```
Readonly
type User = { id: number; name: string };
const user: Readonly<User> = { id: 1, name: "Alice" };
user.id = 2; // ❌ Ошибка: нельзя изменить
```

```
as const
const config = {
  api: "https://api.example.com",
  retry: 3,
  headers: { "Content-Type": "application/json" }
} as const;

// typeof config.headers["Content-Type"] === "application/json"
```

- ✓ `Readonly<T>` делает все свойства только для чтения.
- ✓ `as const` сохраняет литеральные типы и делает объект глубоко `readonly`.
- ✓ Полезно для конфигов и констант.



Итог

- ✓ Типы делают код безопаснее и понятнее.
- ✓ Продвинутые типы помогают создавать гибкие API.
- ✓ Используйте утилиты — они экономят время.
- ✓ Дискриминирующие объединения упрощают логику.
- ✓ Типовые защиты повышают надёжность рантайма.
- ✓ TypeScript отлично работает с современным JavaScript.



Полезные ссылки



Документация TypeScript
<https://www.typescriptlang.org/docs/>



Handbook (RU)
<https://typescriptbook.ru/>



Утилитарные типы
<https://www.typescriptlang.org/docs/handbook/utility-types.html>



Playground
<https://www.typescriptlang.org/play>



Awesome TypeScript
<https://github.com/dzhami/awesome-typescript>

Ещё больше возможностей TypeScript

TypeScript постоянно развивается и даёт ещё больше инструментов для написания чистого, надёжного и масштабируемого кода.



Типы данных

- string
- number
- boolean
- null
- undefined
- symbol
- bigint
- object
- array
- tuple
- enum
- any
- unknown
- never



unknown безопаснее any — перед использованием его нужно сузить или привести к нужному типу.



Дженерики (Generics)

Позволяют создавать универсальные компоненты и функции.

```
function identity<T>(value: T): T {
  return value;
}

const num = identity<number>(123); // number
const str = identity<string>("hi"); // string

class Box<T> {
  private value: T;
  constructor(value: T) {
    this.value = value;
  };
  getValue(): T {
    return this.value;
  }
}

const box = new Box<string>("TypeScript");
console.log(box.getValue());
```



Дженерики повышают гибкость и переиспользуемость кода без потери безопасности типов.



Enum (перечисления)

Задают набор именованных констант.

```
enum Direction {
  Up,
  Down,
  Left,
  Right
}

function move(dir: Direction) {
  switch (dir) {
    case Direction.Up:
      return "↑";
    case Direction.Down:
      return "↓";
  }
}

move(Direction.Up); // "↑"
```



Enum полезны для фиксированных наборов значений и делают код более читаемым.



Вывод типов (Type Inference)

TypeScript сам определяет типы в большинстве случаев.

```
let count = 5; // number
const message = "Hi"; // string
const isActive = true; // boolean
const arr = [1, 2, 3]; // number[]
const user = { name: "Alice", age: 20 }; // { name: string; age: number }
```



Явные типы нужны только тогда, когда это повышает читаемость и предотвращает ошибки.



Приведение типов (Type Assertions)

Говорит компилятору: «доверься мне, я знаю, что делаю».

```
const input = document.getElementById("name") as HTMLInputElement;
input.value = "Alice";

const data = someValue as unknown as MyType;
```



Используйте с осторожностью — можно обойти проверку типов и получить ошибки в рантайме.



Продвинутые типы

Union (объединение)

```
type ID = string | number;

function printId(id: ID) {
  console.log(id);
}
```

Intersection (пересечение)

```
type A = { name: string };
type B = { age: number };
type C = A & B;

const person: C = { name: "Bob", age: 30 };
```

Literal Types (литеральные типы)

```
type Size = "sm" | "md" | "lg";

const size: Size = "md";
```

Readonly

```
type User = {
  readonly id: number;
  readonly name: string;
};
```

Partial / Required

```
type User = { id: number; name: string };
type PartialUser = Partial<User>; // все опционально
type RequiredUser = Required<User>; // все обязательно
```

Pick / Omit

```
type User = { id: number; name: string; email: string };
type UserPreview = Pick<User, "id" | "name">;
type UserWithoutEmail = Omit<User, "email">;
```



Эти утилиты помогают работать со сложными структурами данных.



Модули и пространства имён

Модули (export / import)

```
// math.ts
export function add(a: number, b: number) {
  return a + b;
}

// app.ts
import { add } from "./math";
console.log(add(2, 3));
```

Пространства имён (namespace)

```
namespace Utils {
  export function format(value: string) {
    return value.trim().toLowerCase();
  }
}

const result = Utils.format(" HELLO "); // "hello"
```



Модули — предпочтительный способ организации кода. Namespaces используются реже (обычно в старых проектах).



Настройка tsconfig.json

Основные опции:

```
{
  "compilerOptions": {
    "target": "ES2020",
    "module": "ESNext",
    "lib": ["ES2020", "DOM"],
    "strict": true,
    "esModuleInterop": true,
    "forceConsistentCasingInFileNames": true,
    "skipLibCheck": true,
    "outDir": "dist",
    "rootDir": "src",
    "resolveJsonModule": true
  },
  "include": ["src"]
}
```



Всегда включайте "strict": true — это делает код безопаснее и предсказуемее.



Распространённые ошибки



Type 'string' is not assignable to type 'number'.
Присваивание значения неверного типа.



Property 'name' does not exist on type 'User'.
Обращение к несуществующему свойству.



Function lacks ending return statement and return type does not include 'undefined'.
Функция не возвращает значение.



Object is possibly 'null' or 'undefined'.
Возможное null или undefined.



Type 'undefined' is not assignable to type 'string'.
Использование возможно undefined значения.



Читайте сообщения ошибок внимательно — они помогают понять, что именно не так и как исправить.



Лучшие практики



Используйте строгий режим и избегайте any.



Предпочитайте интерфейсы для объектов.



Используйте типы вместо дублирования логики.



Пишите небольшие и переиспользуемые типы.



Документируйте сложные типы с помощью JSDoc.



Регулярно обновляйте зависимости.



Используйте линтер (ESLint) и форматтер (Prettier).



Пишите тесты вместе с типами.



Полезные ресурсы



Официальный сайт TypeScript
www.typescriptlang.org



Документация TypeScript
www.typescriptlang.org/docs



GitHub
microsoft.com/TypeScript



TypeScript Deep Dive
basarat.github.io



Курсе TypeScript
Academind